Automated Bootstrapping Placement for Encrypted Inference

CS5491 Artificial Intelligence - Milestone draft M-S3-v1

Member 1: [Confirmed English name] | Student ID: [ID]
Member 2: [Confirmed English name] | Student ID: [ID]
Member 3: [Confirmed English name] | Student ID: [ID]
Contact email: [Email]

Public content and formatting draft. Formal identities and the online repository URL are pending; no course submission has occurred.

1. Problem and motivation

We optimise the supplied MLP and attention blocks, including their LayerNorm and Softmax computations, under the course handout's fitted latency model. Each original operator consumes an integer number of ciphertext levels. Expensive bootstrapping restores levels, while free Modswitch operations can lower them. Fan-out and residual connections make placement a graph problem: several consumers may read one refreshed tensor and share its cost. Our objective is total estimated inference latency, including ordinary operators and distinct inserted operations.

The benchmark contains 30 separate instances: each block at every input level L_init from 0 to 14, with L_max fixed at 14. There is no required output level. All original operators, edges and ciphertext counts are preserved. At a merge, the operator uses the minimum incoming level. Bootstrap1 (B1) restores level 14 and costs 1040.15 ms per ciphertext. Bootstrap2 (B2) restores level 13 and costs 1293.13 ms per pair; it is allowed only on tensors of 8 or 16 ciphertexts. Modswitch strictly lowers a level and costs zero.

2. Method: shared-refresh CP-SAT

Our contribution is the shared-refresh formulation, canonical DAG reconstruction and validation workflow. Google's OR-Tools CP-SAT supplies the general search engine. For each original node v, we choose integer input and output levels a_v and b_v, enforce its valid input range and b_v = a_v - d_v, and fix each source input to L_init. For every edge (u,v), we choose a refresh mode r_uv in {0,1,2} and a delivered level t_uv. Without refresh, t_uv <= b_u; B1 permits t_uv <= 14; B2 permits t_uv <= 13 and is disabled on ineligible tensors. At every non-source node, a_v is the minimum delivered level over its predecessors.

A Boolean activation for each producer and refresh type is the logical OR of its outgoing refresh choices. We minimise all original operator costs plus active refresh costs, charging a shared refresh once. Ordinary costs depend on input level and are charged per operator. Coefficients are scaled by 10000, preserving the handout's four-decimal precision exactly. A conservative magnitude bound is checked below 2^53, and both solver gap tolerances are zero. The verified teacher placement supplies a feasible upper bound and search hints.

3. Reconstruction and correctness

The decisions represent exactly the six permitted chains: empty, B1, B2, M, B1-M and B2-M. A Modswitch is inserted only when the delivered level is strictly below the selected available level. Reconstruction keys refresh nodes by producer and type, and switch nodes by producer, refresh mode and target level. Equal prefixes become one shared node. Both refresh types may coexist after a producer when different consumers need them.

Conversely, each permitted chain fixes a refresh choice and delivered level in the model's domains. Its shared refresh charge equals the cost of the reconstructed node. Zero-cost switches affect feasibility and appear in the DAG even though they need no objective activation. This correspondence supports interpreting a matching CP-SAT objective and bound as optimality for the encoded handout problem.

For every solution, an independent audit checks original-graph preservation, ciphertext counts, permitted chains, shared prefixes, integer levels, operator domains, strict drops, merge levels and B2 eligibility. It recomputes node costs with exact decimals. The unchanged teacher scorer then scores the reconstructed DAG; we compare all node and edge levels, counts, depth, budget and latency. The float/decimal cost tolerance is 1e-7 ms. Saved graphs and summaries are rechecked without another optimisation search.

The S3 test run passed 44 checks: the existing 28 and 16 added coverage, evidence and failure-path checks. Adversarial cases include missing or duplicated inputs, changed baselines or saved edges, invalid timings, false improvement flags and fabricated summary success. Fault injection confirms that 30 solver errors produce 30 error records and a failed run. This injected test is separate from the successful course experiment.

Six tiny graphs were completely enumerated independently of CP-SAT and reconstruction. Their minima agree with hand arithmetic, CP-SAT, reconstructed DAGs, the independent audit and the teacher scorer. They cover individual and paired refreshes, shared refresh/switch prefixes, minimum-level merging, different drops after a shared refresh, and changing ciphertext counts. The largest example scores 33280 complete choices; enumeration is restricted to tiny graphs.

4. Experiment and reproducibility

Run S3-OPT-001 executes all 30 instances with shared-refresh-cp-sat-v1 in one configuration: CPython 3.13.9, OR-Tools 9.15.6755, one CPU search worker, seed 5491, and a 60-second limit per instance. The 19 dependencies are pinned. The source commit at execution was 415f2a91462fb361782b84756a552ad85f44a7b5, with a clean working tree. Per-file hashes, configuration, dependencies and timestamps are retained in run.json.

The unchanged S1-BL-001 baselines were calibrated across all 30 inputs. Their costs are 28129.6897 + 0.16*L_init ms for MLP and 53558.1718 ms for attention. Each solution is compared against the baseline for the same block and input. Records retain the inserted DAG, decisions, levels, node costs, solver status/bound and actual CPU resource observations. Running, verification, summary and plotting commands are in the repository README.

5. Preliminary full-instance results

All 30 cases are independently valid, strictly improve on their baselines, and return OPTIMAL with matching bounds. No actual course case failed, timed out or used a baseline fallback. All 18 overlapping scores agree exactly with S2-OPT-001. The 12 newly evaluated attention inputs complete the required range. These are S3 stage results, before the frozen S4 experiment.

Block, inputs	Latency (ms)	Reduction (%)	B1 / B2
MLP, 0-13	26069.9524	7.3223-7.3291	4 / 4
MLP, 14	21911.5924	22.1113	0 / 4
Attention, 0-5	45698.2664	14.6755	10 / 8
Attention, 6-14	41537.6664	22.4438	6 / 8

Table 1. Equal-cost ranges compress the 30 individual records. B1 counts ciphertexts; B2 counts pairs. Their sum is the bootstrap count; each shared operation is charged once. MLP depth is 26 and attention depth is 62. Budget = 14*B1 + 13*B2 + L_init; utilisation = depth/budget. Individual metrics and placements are saved in case JSON and summary.csv.

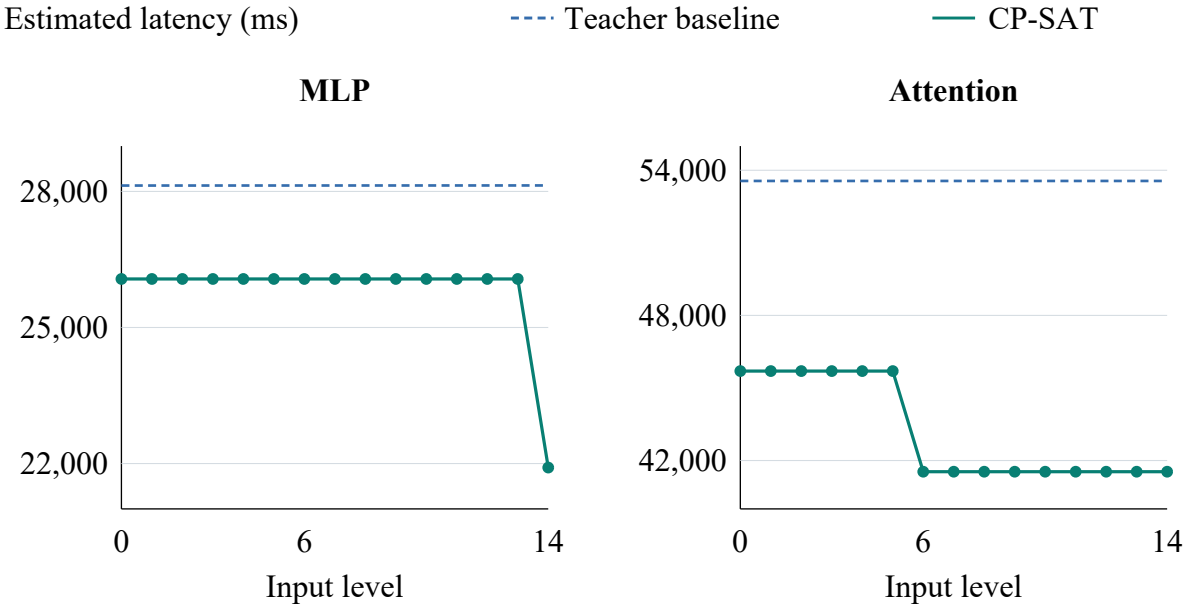


Figure 1. Estimated latency for all 30 input conditions. Points come from saved optimisation results; lines connect integer inputs for readability. These are fitted inference costs, not encrypted-inference measurements or solver times. Source: S3-OPT-001, summary.csv.

Measured solve calls sum to 10.6445 seconds. MLP calls range from 0.0075 to 0.0132 seconds; attention calls range from 0.5166 to 0.9656 seconds. Maximum sampled process RSS is 118.36 MiB. Cases also record CPU time, a separate model/search/validation interval and Windows' process-lifetime peak working set. RSS sampling every 20 ms includes imported libraries and is not an exact per-instance peak. These are observations from one local Windows AMD64 run.

6. Interpretation and representative placements

Attention changes between input levels 5 and 6. Comparing saved level-0 and level-6 solutions shows that their only difference in nonzero node costs is the early B1 after the block input: four ciphertexts cost 4160.60 ms. At input 6, the saved Q and K PCMM1 operators receive level 6 and output level 4 for CCMM1, permitting the observed solution to omit that refresh. Lower latency persists through input 14. Complete searches and matching bounds establish the values for every instance; the representative comparison explains the cost change.

At MLP input 14, the saved optimum also omits an early B1 on four ciphertexts. Relative to input 0, this saves 4160.60 ms while Conjugate X costs 2.24 ms more. Other nonzero node costs agree, explaining the net 4158.36 ms difference. For inputs 0 through 13, optimal estimated latency is constant while the same-input baseline increases slightly.

Bootstrap count alone would give the wrong objective. At attention input 0, our B1/B2 counts are 10/8, totalling 18, versus the baseline's 9/8, totalling 17. Nevertheless, estimated latency falls by 14.6755% because level-dependent ordinary-operation costs matter too. The objective must include them together with shared refresh costs.

Complete scalable DAGs for MLP input 14 and attention input 0 label every original and inserted operator and every edge's level and ciphertext count. They also report latency, counts, depth, budget and utilisation. Adjacent data files retain the exact drawn nodes and edges. These large diagrams support zoomed inspection; Figure 1 gives the report-scale comparison. Different zero-cost switches may yield equally optimal DAGs, so diagrams are tied to saved cases rather than a later solver tie.

7. Limits and remaining work

Optimality applies to the supplied separate graphs, insertion rules and fitted costs. It does not establish actual FHE speed, performance on other networks, or runtime guarantees on teammates' laptops. We did not train a model, run a real encryption library, use a GPU, or optimise the optional connected attention-plus-MLP graph. Current machine checks do not constitute independent clean-environment reproduction.

S4 will freeze the method and full experiment configuration, validate CPU resource observations and produce checked placements for at least two input levels per module. S5 will complete and consolidate course reports and supporting content. S6 will independently reproduce the project and check delivery. Existing S1/S2 experiments and the Proposal retain their historical versions.

Formal English names, student IDs and one contact email must be completed in the restricted submission copy, followed by a fresh content and format check. The public repository URL and grading revision are pending. Instructor-code redistribution permission must be resolved before publishing the bundled reference file. These remain delivery requirements; a local draft does not satisfy the online-repository or actual-submission requirements.

Sources: CS5491 Course Project instructions and Automated Bootstrapping Placement, Sections 2 and 4.1-4.5, supplied course snapshot; unmodified dag_builder.py; OR-Tools CP-SAT 9.15.6755. Source attribution, rules and reproduction commands are included in the repository documentation.